

DATABASE MANAGEMENT SYSTEMS

1. What is Normalization?

Normalization is a systematic database design technique used to organize data in a database. It involves decomposing large tables into smaller, well-structured, related tables to **eliminate data redundancy** and prevent **data anomalies** during insert, update, and delete operations.

★ Core Definition

Normalization is the process of organizing data into multiple related tables to reduce redundancy and maintain data integrity.

Main Objectives:

- Reduce data duplication and storage overhead.
- Eliminate **Insertion**, **Deletion**, and **Updation Anomalies**.
- Ensure data consistency and enforce database integrity constraints.
- Simplify query structure and make the database easier to maintain.

2. What is Data Redundancy?

Data Redundancy refers to the unnecessary repetition of the same data across multiple rows or tables within a database.

Example of Redundant Data:

SID	S_Name	Course_ID	Course_Name	Faculty_ID	Faculty_Name	Salary
1	Ravi	C1	DBMS	F1	Amit	30,000
2	Varun	C2	OS	F2	Rahul	35,000
3	Amrit	C1	DBMS	F1	Amit	30,000
4	Raj	C1	DBMS	F1	Amit	30,000

Observation: Details about **Course C1** (DBMS), **Faculty F1** (Amit), and their salary (30,000) are unnecessarily repeated multiple times.

3. Types of Duplication

A. Row-Level Duplication

Occurs when an entire row/record is identical to another row.

SID	Name	Age
1	Ravi	20

SID	Name	Age
2	Amit	21
1	Ravi	20

Prevention: Enforcing a **Primary Key** (Unique + NOT NULL) completely prevents row-level duplication because duplicate identification keys are disallowed.

B. Column-Level / Attribute-Level Redundancy

Occurs when rows have distinct primary keys, but specific attribute groups (e.g., Course details or Faculty details) repeat unnecessarily across multiple rows.

Note: A Primary Key alone cannot stop attribute-level redundancy. This is precisely what **Normalization** solves.

4. What is an Anomaly?

An **Anomaly** is an unexpected side-effect or data integrity issue that occurs during database manipulation (INSERT, DELETE, or UPDATE) due to poor table design and redundancy.

Memory Trick

IDU = Insertion Anomaly → Deletion Anomaly → Updation Anomaly

5. The Three Major Database Anomalies

1. Insertion Anomaly

Occurs when you cannot insert new data into a table without also adding unrelated or missing information.

Scenario: Suppose a university introduces a new course (C10, MBBS), but no student has enrolled yet. Since SID is the primary key and cannot be NULL, we cannot insert the new course info without generating fake/dummy student data.

2. Deletion Anomaly

Occurs when deleting a specific record unintentionally causes the permanent loss of other essential, unrelated information.

Scenario: If Student SID = 2 (Varun) drops out and we delete his row, we accidentally erase the only existing record for Course C2 (OS) and Faculty F2 (Rahul) from the database.

3. Updation Anomaly

Occurs when updating repeated data in one location requires manual updates across multiple rows, increasing the risk of data inconsistency.

Scenario: If Faculty F1 gets a salary increment from 30,000 to 40,000, every row containing F1 must be modified. Updating only some rows leads to contradictory salary entries in the same database.

6. Solving Anomalies via Table Decomposition

Normalization resolves these issues by decomposing one bloated table into smaller, well-defined related tables linked by foreign keys.

Normalized Structure:

1. Student Table

SID	S_Name
1	Ravi
2	Varun
3	Amrit

2. Course Table

CID	Course_Name
C1	DBMS
C2	OS
C10	MBBS

3. Faculty Table

FID	Faculty_Name	Salary
F1	Amit	30,000
F2	Rahul	35,000

✓ Benefits of Decomposition

- **Insertion:** Course C10 can be added directly into the Course table.
- **Deletion:** Removing a student row leaves course/faculty records untouched.
- **Update:** Updating F1's salary requires modifying exactly one record in the Faculty table.

7. Redundancy to Normalization Flowchart

Poor Database Design



Data Redundancy



Data Anomalies (IDU)



Normalization (Table Decomposition)



Data Consistency & Integrity

⚠ Important Exam Trap

Primary Key vs. Normalization: A Primary Key only prevents duplicate identification values (row-level duplication). It does *not* eliminate structural data redundancy across non-key columns. Normalization decomposes tables to eliminate functional dependency redundancy and anomalies.

8. Quick Revision & Key Points ★

1. **Normalization** decomposes large tables into smaller tables to reduce redundancy and maintain integrity.
2. **Three Anomalies:** Insertion (can't add without missing data), Deletion (unintended loss of data), Update (inconsistency during updates).
3. **Primary Key** ensures unique identification of rows but doesn't prevent attribute-level redundancy.
4. **Normal Forms Sequence:** 1NF → 2NF → 3NF → BCNF → 4NF → 5NF.

 Super-Easy Memory Trick: REDUCE

- R** → Reduce redundancy
- E** → Eliminate anomalies
- D** → Divide / decompose tables
- U** → Update consistently
- C** → Consistency
- E** → Efficient database design